

JavaAgent CLI 伪代码文档

本文档用自然语言描述整体架构和核心流程，不包含任何可运行代码。

包结构总览

com.javagent

- └─ JavaAgentCLI # 程序入口
- └─ BannerPrinter # 方框风格启动横幅
- └─ SlashCommandCompleter # 斜杠命令 Tab 自动补全
- └─ core/ # 核心逻辑
 - └─ Agent # Agent 引擎
 - └─ Config # 配置管理
 - └─ ConversationManager # 对话管理
 - └─ ApprovalManager # 审批管理
 - └─ ApprovalHandler # 审批回调接口
 - └─ ApprovalDecision # 审批决策枚举
 - └─ ApprovalOutcome # 审批结果
 - └─ ToolStats # 工具执行统计
- └─ model/ # 数据模型
 - └─ Role # 角色枚举
 - └─ Message # 消息记录
 - └─ ToolCall # 工具调用请求
 - └─ ToolResultMessage # 工具结果消息
 - └─ ModelResponse # 模型响应
 - └─ ModelClient # 模型客户端接口
 - └─ MockModelClient # 模拟客户端
 - └─ OpenAiCompatibleModelClient # OpenAI 客户端
 - └─ TextStreamHandler # 流式输出接口
 - └─ ToolDisplayCallback # 工具执行显示回调
- └─ tools/ # 工具集
 - └─ Tool # 工具接口
 - └─ ToolDefinition # 工具定义
 - └─ ToolExecutionResult # 工具执行结果
 - └─ ToolRegistry # 工具注册中心
 - └─ ToolProvider # SPI 插件接口
 - └─ FileToolSupport # 文件工具辅助
 - └─ ReadFileTool # 读文件
 - └─ GrepTool # 文本搜索
 - └─ ListDirectoryTool # 目录列表
 - └─ WriteFileTool # 写文件
 - └─ EditTool # 精确文本替换

- | | — DeleteFileTool # 删除文件
- | | — BashTool # Shell 命令
- | | — NetworkTool # HTTP 请求
- | — util/ # 工具包
- | — Terminal # ANSI 终端颜色
- | — MarkdownRenderer # Markdown 渲染
- | — Sanitizer # 敏感信息过滤
- | — RateLimiter # 速率限制

一、com.javagent 包

JavaAgentCLI —— 程序入口

作用

核心字段

- 配置对象、对话管理器、工具注册中心、模型客户端、Agent 引擎
- JLine3 LineReader (终端行编辑、历史翻页)
- AtomicBoolean awaitingApproval (审批时暂停 spinner)

方法流程

main

- 创建 JavaAgentCLI 实例，调用 run 方法

run

- 加载默认配置 (从 config.properties 文件)
- 解析命令行参数 (--mock、--real、--api-key、--help)
- 初始化/重建运行时组件
- 启动命令行交互循环

applyArgs

- 遍历命令行参数数组
- 遇到 --mock 则切换为模拟模式
- 遇到 --real 则切换为真实模式
- 遇到 --api-key 则设置 API 密钥
- 遇到 --help 则打印帮助并退出

rebuildRuntime

- 如果对话管理器为空则创建新实例 (已存在则复用)
- 创建工具注册中心，依次注册：读文件、搜索、目录列表、写文件、编辑、删除文件
- 如果 Bash 已启用，注册 Bash 工具
- 注册 Network 工具
- 通过 SPI (ServiceLoader) 加载外部 ToolProvider 插件
- 根据当前模式选择模型客户端 (模拟模式用 Mock，真实模式用 OpenAI 兼容客户端)
- 创建 Agent 实例，传入配置、模型客户端、工具注册中心、对话管理器

startCli

- 打印方框风格启动横幅 (BannerPrinter)
- 显示当前模式、模型、工具数量
- 尝试恢复上次的会话
- 进入主循环:
- 显示彩色提示符
- 读取用户输入 (JLine3 LineReader)
- 如果输入为空则跳过
- 如果以
- 否则交给 Agent 处理, 传入审批回调和流式输出处理器
- 根据流式输出是否已触发, 决定直接打印还是换行

handleCommand

- 解析命令和参数
- /help
- /exit
- /clear
- /save
- /load
- /sessions
- /tools
- /mode
- /stream
- /bash
- /bypass
- /prompt
- /approvals
- /reload
- /export
- /stats
- /network
- /status

promptApproval

- 暂停 Braille spinner (awaitingApproval.set(true))
- 打印工具调用信息 (工具名、参数)
- 提示用户输入 yes/no/cancel
- 根据用户输入返回对应的审批决策
- 恢复 Braille spinner (awaitingApproval.set(false))

ConsoleTextStreamHandler (内部类)

- 实现 TextStreamHandler 接口
- 每收到一块文本就立即打印到控制台
- 记录是否已输出过文本, 用于判断是否需要额外换行

二、com.javagent.core 包

Agent —— 核心引擎

作用

核心字段

- 配置、模型客户端、工具注册中心、对话管理器、审批管理器、工具统计

processTurn (无流式)

- 调用带流式参数的版本，传入 null

processTurn (核心方法)

- 把用户消息加入对话历史
- 执行上下文压缩
- 构建系统提示词 (告诉 AI 它是谁、有哪些工具可用、编辑工具使用规范)
- 进入 Agent Loop (最多循环 maxIterations 次) :
- 根据配置决定是否启用流式输出
- 调用模型客户端发送请求，获取响应
- 如果是错误响应:
- 如果错误包含 "reasoning_content", 自动清理旧上下文并重试
- 记录错误消息，自动保存，返回错误文本
- 如果是纯文本响应 → 记录 AI 回复 (含 reasoningContent) , 自动保存，返回回复文本
- 如果是工具调用响应 → 记录 AI 的工具调用消息，逐个执行工具调用:
- 调用 displayCallback.onToolStart 通知 UI
- 执行工具，记录 ToolStats 统计
- 同一工具连续失败 3 次 → 中断循环
- 工具结果经 Sanitizer 脱敏后加入对话历史
- 调用 displayCallback.onToolEnd 通知 UI
- 继续循环
- 超过最大循环次数 → 记录提示消息，自动保存，返回提示

executeToolCall

- 从工具注册中心查找工具，找不到则返回错误
- 通过审批管理器检查是否被授权
- 审批不通过则返回拒绝原因
- 审批通过则执行工具，记录统计，返回执行结果

buildSystemPrompt

- 设定 AI 身份 ("JavaAgent CLI, 简洁的编程助手")
- 添加使用原则 (按需使用工具、工作区内操作、避免破坏性操作)
- 添加编辑工具使用规范 (old_string 必须精确匹配)
- 如果有自定义提示词则追加
- 列出所有可用工具的名称、描述、别名和必填参数

autoSaveQuietly

- 如果配置了自动保存，则保存当前会话

- 保存失败时静默忽略（写日志），不影响主流程

Config —— 配置管理器

作用

配置项

配置项

说明

默认值

agent.mock_mode

是否使用模拟模式

true

agent.api_key

API 密钥

空

agent.base_url

API 基础 URL

https://api.openai.com/v1

agent.model

模型名称

gpt-5.4-mini

agent.auto_save

是否自动保存会话

true

agent.max_iterations

最大工具调用循环次数

12

agent.enable_bash

是否启用 Bash 工具

false

agent.stream_responses

是否流式输出

true

agent.system_prompt

自定义系统提示词

空

agent.approval_cache

是否启用审批缓存

true

agent.allow_external_paths

是否允许访问工作区外路径

false

agent.bypass_permissions

是否跳过所有工具审批确认

false

agent.max_context_messages

最大上下文消息数量

100

agent.rate_limit_qps

API 请求速率限制（每秒请求数）

10

方法流程

loadDefault

- 工作目录设为当前目录，应用主目录设为 ~/javaagent-cli
- 调用内部加载方法

load

- 从文件读取配置 → 填充缺失的默认值 → 保存回文件

配置文件查找顺序

1. 项目根目录（有 pom.xml 的目录）
2. 工作目录
3. ~/javaagent-cli/config.properties

路径方法

- stateDirectory: 状态目录（优先在工作目录下创建 javaagent-cli，否则放 appHome）
- sessionPath: 旧版会话文件路径
- sessionDirectory: 会话存储目录
- currentSessionMarkerPath: 当前会话标记文件

ConversationManager —— 对话管理器

作用

核心字段

- 配置对象、Jackson 序列化器
- 当前会话 ID、标题、消息列表、开始时间、最后一条 AI 回复

方法流程

构造器

- 保存配置引用
- 初始化 Jackson 序列化器（支持 Java 时间类型、日期用字符串、格式化输出）
- 开始一个新的空会话

addUserMessage / addAssistantMessage / addAssistantToolCallMessage /
addToolResultMessage

- 分别创建对应角色的消息对象，加入当前消息列表

compactIfNeeded

- 如果消息数量超过 maxContextMessages，压缩旧消息
- 保留系统提示词和最近的消息

currentContext

- 返回当前消息列表的不可变副本

startNewSession

- 生成新的会话 ID (UUID)
- 设置标题 (传入标题或默认标题)
- 清空消息列表
- 重置会话开始时间和最后回复

saveCurrentSession

- 创建会话快照 (包含 ID、标题、开始时间、更新时间、消息列表、最后回复)
- 确保目录存在
- 保存到三处: 会话目录下的 JSON 文件、旧版路径、当前会话标记文件

loadLastSession

- 优先读取标记文件, 获取会话 ID
- 如果找到对应的会话文件则加载
- 否则回退到旧版路径
- 加载后应用快照恢复会话状态

loadSession

- 列出所有保存的会话
- 按 ID 或标题模糊匹配
- 如果是 "latest" 则加载最新的
- 找到则加载快照并更新标记文件

listSessions

- 遍历会话目录下的 JSON 文件
- 提取每个会话的摘要信息
- 按更新时间倒序排列

exportToMarkdown

- 将当前对话导出为 Markdown 格式

内嵌数据结构

- SessionSummary: 会话摘要 (ID、标题、更新时间、消息数)
- SessionSnapshot: 会话快照 (完整会话数据, 用于保存/加载)

ApprovalManager —— 审批管理器

作用

核心字段

- 配置对象、审批缓存 (Map 结构)

authorize

- 如果 bypass 模式开启 → 直接返回批准
- 获取工具定义
- 执行策略检查 (evaluatePolicy)
- 如果策略结果为"允许" → 直接返回批准

- 如果策略结果为"拒绝" → 直接返回拒绝
- 如果策略结果为"需审批" → 检查缓存
- 缓存命中 → 沿用缓存的审批决策
- 缓存未命中 → 调用审批回调请求用户确认
- 用户决策不为"取消"时存入缓存
- 返回审批结果

evaluatePolicy (策略检查)

- 如果是 Bash 工具且未启用 → 拒绝
- 提取工具参数中的路径, 如果路径在工作区外且不允许外部路径 → 拒绝
- 如果是破坏性工具且路径为受保护路径 (.git、javaagent-cli 等) → 拒绝
- 如果是删除工具且目标是工作区根目录或目录 → 拒绝
- 如果工具标记为不需要审批 → 允许
- 其他情况 → 需要用户审批

extractPath

- 从工具参数中提取路径 (大多数工具用 "path" 参数, Bash 用 "workingDirectory")
- 将路径解析为绝对路径

isProtectedPath

- 检查路径是否位于状态目录、配置文件、.git 目录、javaagent-cli 目录等受保护位置

cacheKey

- 生成缓存键: 工具名 + 标准化后的参数 (路径参数会解析为绝对路径后排序)

ApprovalDecision —— 审批决策枚举

作用

- APPROVED: 同意执行
- DENIED: 拒绝执行
- CANCELLED: 取消 (中止操作)

ApprovalOutcome —— 审批结果

作用

工厂方法

- approved / cachedApproved: 创建批准结果 (区分是否来自缓存)
- denied / cachedDenied: 创建拒绝结果 (区分是否来自缓存)

ApprovalHandler —— 审批回调接口

作用

- 唯一方法 request(ToolCall): 接收工具调用请求, 返回审批决策
- 标记为函数式接口, 可用 Lambda 创建

ToolStats —— 工具执行统计

作用

核心数据结构

record

- 记录一次工具执行（工具名、耗时、是否出错）

format

- 格式化输出统计表格

StatsEntry

- calls: 调用次数
- totalMs: 总耗时
- errors: 错误次数
- avgMs: 平均耗时

三、com.javagent.model 包

Role —— 角色枚举

作用

- SYSTEM: 系统提示词，告诉 AI 如何行为
- USER: 用户输入的消息
- ASSISTANT: AI 的回复
- TOOL: 工具执行后返回的结果

每个枚举值持有 apiValue 字段，对应 API 协议中的小写字符串（如 "user"）。

Message —— 消息记录

作用

字段

约束

工厂方法

- system / user / assistant: 创建纯文本消息
- assistantWithToolCalls: 创建包含工具调用的 AI 消息
- toolResult: 创建工具结果消息（区分成功和失败）

辅助方法

- hasToolCalls: 是否包含工具调用
- isToolError: 工具执行是否出错

ToolCall —— 工具调用请求

作用

字段

工厂方法

ToolResultMessage —— 工具执行结果消息

作用

字段

工厂方法

ModelResponse —— 模型响应

作用

响应类型

- TEXT: 纯文本回复
- TOOL_CALLS: 请求调用工具
- ERROR: 请求出错

字段

工厂方法

辅助方法

ModelClient —— 模型客户端接口

作用

方法

- chat (核心) : 发送对话请求, 返回模型响应
- chat (带流式输出) : 默认实现, 先获取完整响应, 再将文本分块发送给流式处理器
- name: 返回客户端名称

分块策略

MockModelClient —— 模拟模型客户端

作用

chat

- 如果消息列表为空 → 返回就绪提示
- 如果最后一条是工具结果 → 总结工具执行结果
- 找到最后一条用户消息
- 关键词匹配:
 - "删除/delete" → 返回 delete_file 工具调用
 - "写入/write" → 返回 write_file 工具调用
 - "编辑/edit/修改/替换" → 返回 edit 工具调用
 - "列出/目录/ls" → 返回 list_directory 工具调用
 - "读取/read" → 返回 read_file 工具调用
 - "搜索/grep" → 返回 grep 工具调用
 - "bash/shell" → 返回 bash 工具调用
 - "网络/http/fetch" → 返回 network 工具调用
 - "help/工具" → 返回帮助文本
- 未匹配 → 返回默认文本提示

summarizeToolResult

- 根据工具名称 (read_file、grep、list_directory、edit 等) 生成不同格式的总结
- 截取结果的前若干行

OpenAiCompatibleModelClient —— OpenAI 兼容客户端

作用

chat

- 检查 API Key 是否已配置, 未配置则返回错误

- 应用速率限制 (RateLimiter)
- 构建 JSON 请求体 (含 stream: true)
- 发送 HTTP POST 请求到 API 端点
- 如果启用流式输出 → 逐行解析 SSE 事件流
- 如果未启用流式输出 → 解析完整 JSON 响应
- 检查 HTTP 状态码, 400+ 尝试降级重试 (去除 tools 参数)
- 解析响应, 转换为 ModelResponse
- 如果 API 因 reasoning_content 报错, 返回特定错误信息

buildRequestBody

- 设置模型名、温度、工具选择策略
- 构建消息数组: 系统提示词 + 对话历史
- 根据消息角色分别处理: USER/SYSTEM 只设内容, ASSISTANT 可能含工具调用和 reasoningContent, TOOL 需配对工具调用 ID
- 构建工具定义数组: 每个工具包含名称、描述、参数 Schema、必填参数列表

SSE 流式处理

- 使用 HttpClient.BodyHandlers.ofLines() 逐行读取
- 解析每个 "data:" 行的 JSON
- 提取 delta.content 并实时调用 streamHandler.onChunk()
- 增量解析工具调用的 id/name/arguments (ToolCallDelta)
- 提取 reasoning_content (thinking 模型)
- [DONE] 事件结束流

parseResponse

- 检查响应中是否有错误对象
- 提取第一个 choice 的 message
- 如果 message 中有 tool_calls 字段 → 解析为工具调用响应
- 如果有 reasoning_content → 提取推理内容
- 否则 → 提取文本内容作为纯文本响应

TextStreamHandler —— 流式输出接口

作用

- 函数式接口, 唯一方法 onChunk(chunk)
- 可用 Lambda 创建

ToolDisplayCallback —— 工具执行显示回调

作用

```
public interface ToolDisplayCallback {
    void onToolStart(String toolName, String summary);
    void onToolEnd(String toolName, boolean success, String resultSummary);
    default void onToolEnd(String toolName, boolean success, String resultSummary, String fullContent);
}
```

四、com.javagent.tools 包

Tool —— 工具接口

作用

方法

- definition: 返回工具的元信息定义
- execute: 接收参数 Map, 执行操作, 返回执行结果
- configure(Path): 可选, 配置工作区根目录

ToolDefinition —— 工具定义

作用

字段

- name: 工具名称
- description: 功能描述
- parameterDescriptions: 每个参数的含义
- parameterTypes: 每个参数的类型
- requiredParameters: 必填参数集合
- requiresApproval: 是否需要用户审批
- readOnly: 是否只读
- destructive: 是否破坏性操作
- aliases: 工具别名列表

ToolExecutionResult —— 工具执行结果

作用

工厂方法

ToolRegistry —— 工具注册中心

作用

核心数据结构

register

- 将工具按名称 (小写) 存入名称 Map
- 遍历别名, 逐个存入别名 Map

find

- 先按名称查找
- 找不到再按别名查找
- 都找不到返回空

definitions

- 返回所有工具的定义列表, 用于告诉 AI 有哪些工具

listTools

- 生成格式化的工具列表文本, 包含名称、别名、描述、审批标志

ToolProvider —— SPI 插件接口

作用

```
public interface ToolProvider {  
    List tools();  
}
```

在 META-INF/services/com.javagent.tools.ToolProvider 中注册即可。

FileToolSupport —— 文件工具辅助类

作用

方法

- stringValue: 从 Map 提取字符串值, null 返回空串
- intValue: 从 Map 提取整数值, 兼容 Number 和 String 类型
- booleanValue: 从 Map 提取布尔值, 兼容 Boolean 和 String 类型
- normalizePath: 规范化路径, 去除冗余的 "." 和 ".."
- isBinary: 检测文件是否为二进制文件 (只检测 null 字节)
- checkInsideWorkspace: 检查路径是否在工作区内

ReadFileTool —— 读文件工具

作用

安全标记

安全限制

- 文件大小超过 256KB 拒绝读取
- 二进制文件拒绝读取

execute

- 提取并验证路径参数
- 规范化路径
- 检查文件是否存在且是普通文件
- 检查文件大小和是否为二进制
- 读取文件全部行
- 根据 offset 和 limit 截取指定范围
- 构建带行号的输出格式
- 如果还有更多行, 显示省略提示

GrepTool —— 文本搜索工具

作用

安全标记

限制

execute

- 提取并验证搜索模式
- 提取搜索路径 (默认当前目录)
- 编译正则表达式 (支持大小写敏感/不敏感)
- 如果目标是文件 → 直接搜索该文件

- 如果目标是目录 → 递归遍历，跳过 .git、target 等目录
- 对每个文件逐行匹配，输出匹配行（文件路径:行号:内容）
- 统计扫描文件数和匹配数

ListDirectoryTool —— 目录列表工具

作用

安全标记

execute

- 提取路径参数（默认当前目录）
- 检查路径是否存在且是目录
- 根据是否递归选择遍历方式
- 过滤掉自身、按名称排序、限制条目数
- 输出 [D] 标记目录、[F] 标记文件
- 递归模式显示相对路径，非递归模式只显示文件名

WriteFileTool —— 写文件工具

作用

安全标记

限制

execute

- 提取路径和内容参数
- 检查内容大小
- 规范化路径
- 检查目标不是目录
- 如果文件已存在且是二进制文件，拒绝覆盖
- 创建父目录（如不存在）
- 根据追加/覆盖模式选择写入方式
- 返回写入结果（字符数、路径、模式）

EditTool —— 精确文本替换工具

作用

安全标记

别名

execute

- 提取 path、old_string、new_string 参数
- 检查文件是否存在且是文本文件
- 读取文件全部内容
- 搜索 old_string 的所有匹配位置
- 如果没有匹配 → 尝试模糊查找，提示类似文本位置
- 如果有多个匹配且 replace_all=false → 报错要求更精确的上下文
- 执行替换
- 写回文件

- 构建 diff 预览（红色删除行、绿色新增行）
- 返回编辑统计

DeleteFileTool —— 删除文件工具

作用

安全标记

execute

- 提取路径参数
- 规范化路径
- 检查文件是否存在
- 检查是否是普通文件（不支持删除目录）
- 执行删除
- 返回删除结果

BashTool —— Shell 命令工具

作用

安全标记

安全限制

- 默认禁用，需用户手动开启
- 内置 10 类危险命令检测（正则匹配）
- 默认超时 10 秒
- 输出超过 50,000 字符自动截断
- 跨平台：Windows 用 cmd.exe /c, Unix 用 /bin/bash -lc

execute

- 提取命令参数
- 检查命令是否为危险命令（10 类正则匹配）
- 提取超时参数
- 创建进程构建器（Linux 用 bash, Windows 用 cmd）
- 合并标准错误到标准输出
- 启动进程
- 逐行读取输出，超过限制则截断
- 等待进程结束（带超时），超时则强制终止
- 组装结果（命令 + 输出 + 退出码）
- 退出码为 0 返回成功，否则返回失败

NetworkTool —— HTTP 请求工具

作用

安全标记

别名

execute

- 提取 url、method、timeout 参数
- 检查 URL 是否为空

- 默认方法为 GET，超时为 30 秒
- 创建 HttpClient
- 构建 HttpRequest
- 发送请求，获取 HttpResponse
- 组装结果：状态码、URL、方法、响应头、响应体
- 返回结果

五、com.javagent.util 包

Terminal —— ANSI 终端格式化

作用

方法

MarkdownRenderer —— Markdown 渲染

作用

支持

Sanitizer —— 敏感信息过滤

作用

模式

RateLimiter —— 速率限制

作用

核心流程：Agent Loop 伪代码

用户输入

↓

将用户消息加入对话历史

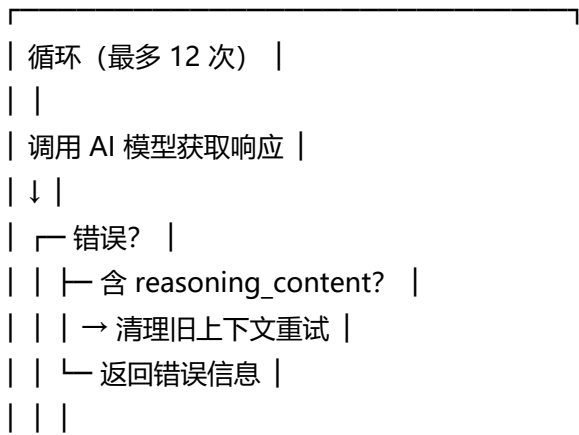
↓

上下文压缩 (compactIfNeeded)

↓

构建系统提示词 (身份 + 工具列表 + 编辑规范)

↓




```
| └─ 纯文本? → 返回文本回复 |
| | |
| └─ 工具调用? |
| ↓ |
| 记录 AI 的工具调用消息 |
| ↓ |
| 对每个工具调用: |
| 1. 通知 UI 开始执行 |
| 2. 从注册中心查找工具 |
| 3. 审批检查 (含 bypass) |
| 4. 执行工具 |
| 5. 记录 ToolStats 统计 |
| 6. 连续失败 ≥3 → 中断 |
| 7. Sanitizer 脱敏 |
| 8. 将结果加入对话历史 |
| 9. 通知 UI 执行完成 |
| ↓ |
| 继续循环 (让 AI 看到结果) |
└──────────────────────────────────┘
```

↓
超过最大次数 → 返回提示

审批流程伪代码

工具调用请求

↓
bypass 模式开启? → 直接批准

↓
策略检查

```
|─ Bash 未启用? → 拒绝
|─ 路径在工作区外? → 拒绝
|─ 受保护路径? → 拒绝
|─ 删除根目录? → 拒绝
|─ 只读工具? → 自动通过
└─ 其他 → 需要审批
```

↓
检查审批缓存

```
|─ 缓存命中 (之前批准过) → 自动通过
|─ 缓存命中 (之前拒绝过) → 自动拒绝
└─ 缓存未命中
```

↓
暂停 Braille spinner
请求用户确认

- └─ 用户同意 → 批准, 存入缓存
- └─ 用户拒绝 → 拒绝, 存入缓存
- └─ 用户取消 → 拒绝, 不缓存

恢复 Braille spinner

安全机制总览

安全层级:

1. 工作区隔离 — 所有文件操作限制在项目目录内
2. 审批确认 — 写文件、删文件、编辑、bash、网络请求需要用户确认
3. Bypass 模式 — 可选跳过所有审批 (后果自行承担)
4. 危险命令检测 — 10 类正则匹配 (rm -rf、fork bomb、reverse shell 等)
5. 敏感信息脱敏 — API Key/Token 自动过滤, 防止泄漏到会话文件
6. 连续失败保护 — 同一工具连续失败 3 次自动中断
7. 速率限制 — 令牌桶算法控制 API 请求频率
8. 上下文压缩 — 防止上下文超限
9. API 自动重试 — 最多 3 次重试, 指数退避
10. 工具降级 — HTTP 400 时自动去除 tools 参数重试